

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5115

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: 2

Total Marks: 50

Annexure A

Code of Conduct:

I Nalluri Charan Sai (192372003) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

N Charan Sai

Annexure A

Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

- A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
- B. Optimize the code to reduce execution time for large input datasets (≥ 10 MB).
- C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

Solution: A Python program implementing DES and 3DES encryption produces inconsistent ciphertext for identical inputs across multiple runs. (a) Identify and fix logical and implementation errors in key scheduling and permutation steps. (b) Optimize the code to reduce execution time for large input datasets (≥ 10 MB). (c) Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

(a) Identifying and Fixing the Key-Scheduling and Permutation Errors

Inconsistent ciphertext for identical inputs across multiple runs' is a strong symptom pointing to hidden mutable state being carried between calls, rather than a purely mathematical mistake (a purely mathematical bug would produce wrong-but-consistent ciphertext every time, not different ciphertext on different runs). Inspecting a representative round-key generation routine reveals two distinct, compounding defects:

Buggy key-schedule and permutation code:

```
PC1 = [57,49,41,33,25,17,9, 1,58,50,42,34,26,18,
       10,2,59,51,43,35,27,19,11,3,60,52,44,36,
       63,55,47,39,31,23,15, 7,62,54,46,38,30,22,
       14,6,61,53,45,37,29,21,13,5,28,20,12,4]

def generate_round_keys(key_bits, round_keys=[]):    # BUG 1
    permuted = [key_bits[i] for i in PC1]           # BUG 2
    left, right = permuted[:28], permuted[28:]
    for rnd in range(16):
        left = left[SHIFTS[rnd]:] + left[:SHIFTS[rnd]]
        right = right[SHIFTS[rnd]:] + right[:SHIFTS[rnd]]
        combined = left + right
        subkey = [combined[i] for i in PC2]          # BUG 2 (again)
        round_keys.append(subkey)
    return round_keys
```

Bug 1 — Mutable default argument (key-scheduling state leak): `round_keys=[]` is evaluated only once, when the function is first defined, and that same list object is then silently reused as the default on every subsequent call that does not pass its own list explicitly. Each call to `generate_round_keys()` therefore appends its 16 new subkeys onto whatever was left over from the previous call in the same program run, instead of starting from an empty list. The function's returned key schedule keeps growing and shifting with every call, so encrypting the exact same plaintext under the exact same key on a later call

in the same run uses a different, corrupted set of round keys than an earlier call — producing different ciphertext for identical input, exactly as observed.

Bug 2 — Off-by-one indexing in the permutation tables: PC1 and PC2 (and, in the full implementation, the Initial Permutation IP and Expansion table E) are defined in the DES standard using 1-based bit positions (bits numbered 1 through 64). Python lists are 0-indexed, so `key_bits[i]` with `i` taken directly from PC1 silently selects the bit one position to the right of the one intended for every single entry in the table. This is a logical error, not a crash (since the table's values never exceed the list's valid index range), so it produces a systematically wrong — but not obviously erroring — permutation on every call, compounding with Bug 1's stale state to make the output both wrong and inconsistent.

Fixed key-schedule and permutation code:

```
def generate_round_keys(key_bits):
    round_keys = []                # FIX 1: fresh list every call
    permuted = [key_bits[i - 1] for i in PC1]    # FIX 2: 1-based table -> 0-based index
    left, right = permuted[:28], permuted[28:]
    for rnd in range(16):
        left = left[SHIFTS[rnd]:] + left[:SHIFTS[rnd]]
        right = right[SHIFTS[rnd]:] + right[:SHIFTS[rnd]]
        combined = left + right
        subkey = [combined[i - 1] for i in PC2]    # FIX 2 applied consistently
        round_keys.append(subkey)
    return round_keys
```

With both fixes applied, `generate_round_keys()` is now a pure, stateless function of its `key_bits` argument alone: calling it twice with the same key always returns the identical 16-entry key schedule, and every bit selected by PC1/PC2 (and, by the same fix applied throughout the rest of the implementation, IP and E) now maps to the bit the DES standard actually specifies. The same class of off-by-one fix must be applied everywhere else in the implementation that indexes a bit using one of the standard's 1-based permutation tables directly as a Python list index.

(b) Optimizing the Code for Large Input Datasets (≥10 MB)

Even after correctness is restored, a naive DES/3DES implementation that manipulates plaintext as Python lists of individual bit characters is far too slow for multi-megabyte inputs, and the original code compounds this with several avoidable inefficiencies:

Inefficient file-encryption routine:

```
def des_encrypt_file_slow(filename, key_bits):
    with open(filename, 'rb') as f:
        data = f.read()                # loads entire file into RAM
        bitstream = bytes_to_bitlist(data)    # huge Python list of '0'/'1' ints
        ciphertext_bits = []
        for block in chunk(bitstream, 64):
            round_keys = generate_round_keys(key_bits)    # recomputed for EVERY block!
            ciphertext_bits += des_encrypt_block(block, round_keys)
        return bitlist_to_bytes(ciphertext_bits)
```

- Whole-file loading: reading the entire ≥ 10 MB file into memory at once, then converting it into an even larger Python list of individual bit values, multiplies memory usage many times over and prevents the program from starting encryption until the entire file has been read from disk.
- Redundant key-schedule recomputation: `generate_round_keys()` is called again inside the per-block loop, so the same 16-round key schedule is recomputed from scratch for every 64-bit block — for a 10 MB file (roughly 1.25 million blocks) that is 1.25 million unnecessary repetitions of a computation that only needs to happen once per key.
- Bit-list representation: representing every bit as a separate Python list element (rather than working with native bytes/ints and bitwise operators) means every permutation, shift, and XOR touches millions of individual Python objects instead of a handful of machine-word operations, which is dramatically slower than necessary in CPython.
- Growing-list concatenation: `ciphertext_bits += ...` repeatedly extends a Python list; while list append/extend is amortized $O(1)$, converting the final multi-million-element bit list back into bytes in one pass (`bitlist_to_bytes`) is itself a further avoidable full-data-size pass that a byte-oriented implementation would not need at all.

Optimized, streaming file-encryption routine:

```
def des_encrypt_file_fast(in_path, out_path, key_bits, chunk_size=1 << 20):
    round_keys = generate_round_keys(key_bits)    # computed ONCE for the whole
    file

    with open(in_path, 'rb') as fin, open(out_path, 'wb') as fout:
        buf = bytearray()
        while chunk := fin.read(chunk_size):      # stream in 1 MB chunks, bounded
            memory

            buf.extend(chunk)
            n_full_blocks = (len(buf) // 8) * 8
            if n_full_blocks:
                fout.write(des_encrypt_blocks_bytes(buf[:n_full_blocks], round_keys))
                del buf[:n_full_blocks]
            if buf:
                fout.write(des_encrypt_blocks_bytes(pad(buf, 8), round_keys))
```

Four changes account for the bulk of the speed-up: (1) the key schedule is generated exactly once and reused for every block, eliminating the 1.25-million-fold redundant recomputation identified above; (2) the file is streamed in fixed-size chunks (here 1 MB) rather than loaded wholesale, bounding peak memory usage regardless of input size and letting encryption begin before the whole file has finished being read from disk; (3) `des_encrypt_blocks_bytes` operates on raw bytes/integers with bitwise shift, AND, OR, and XOR operators instead of Python lists of bit characters, which CPython executes far faster since each operation works on a machine word rather than iterating per-bit Python objects; and (4) for genuinely large-scale or production use, the hand-written DES loop itself should be replaced with a compiled, C-accelerated `Cipher.DES3`, since a compiled cryptographic primitive is routinely one to two orders of magnitude faster than the equivalent pure-Python bit-manipulation code, regardless of how well the Python is optimized. Where the chosen mode of operation allows independent block processing (ECB, or CTR-style constructions), the block loop can additionally be parallelized across CPU cores using the multiprocessing

module — though, as discussed in Question 2, CBC's block-to-block chaining dependency limits this parallelism specifically during encryption, though CBC decryption can still be parallelized since every ciphertext block is already available up front.

(c) Would Replacing DES/3DES with a Modern Cipher (AES) Improve Both Security and Performance?

Property	DES	3DES (Triple DES)	AES (128/256-bit)
Effective key strength	56 bits — brute-forced in under a day with modern hardware	~112 bits effective (meet-in-the-middle attack) despite a 168-bit nominal key	128 / 256 bits, no practical attack faster than brute force
Standards status	Formally withdrawn / deprecated by NIST	Disallowed for new use by NIST SP 800-131A as of 2023	Current NIST/global standard (FIPS 197)
Structure	16-round Feistel network, heavy bit-level permutation	DES applied 3× sequentially — 48 rounds total	10–14 round substitution-permutation network, byte/word-oriented
Throughput	Slow — bit-level permutations are costly in software	~3× slower than single DES (three full passes)	Fast — byte-oriented S-box/round design suits both software and hardware
Hardware acceleration	None widely available	None widely available	AES-NI instruction set built into virtually all modern CPUs

Yes — replacing DES/3DES with AES improves both security and performance simultaneously, which is unusual, since security and performance often trade off against each other; here the trade-off runs the same direction for both.

Security improvement: DES's 56-bit key space has been comprehensively broken by brute force since 1998 (the EFF's 'Deep Crack' machine) and is now breakable in hours on commodity hardware, while 3DES's ~112-bit effective strength (reduced from its nominal 168 bits by the meet-in-the-middle attack) has been formally disallowed by NIST for new applications since 2023. AES-128 already exceeds 3DES's effective security margin, and AES-256 provides a very large additional safety against any future advances in cryptanalysis or quantum computing (Grover's algorithm only halves AES's effective key length, leaving AES-256 at a quantum-resistant ~128-bit equivalent security level, whereas the same halving would leave 3DES trivially breakable).

Performance improvement: AES's substitution-permutation network operates on whole bytes using table-driven S-boxes, rather than DES's bit-level permutation network, which is inherently far more efficient in both software and hardware. Because AES is the universal modern standard, essentially every modern CPU (x86, ARM) implements the AES-NI/Crypto Extensions instruction set directly in hardware, executing a full AES round in a single instruction — a level of hardware support 3DES has never received. Combined with AES needing only a single encryption pass (versus 3DES's three full DES passes per block), a correctly optimized AES implementation on typical modern hardware substantially outperforms even a well-optimized 3DES implementation, on top of already being dramatically faster than the original buggy, non-hardware-accelerated bit-level DES/3DES code being debugged in this task.

Recommendation: the organization should migrate from DES/3DES to AES (typically AES-256-GCM for new designs, which additionally provides built-in authentication) rather than continuing to invest debugging and optimization effort into a cipher family that is already cryptographically obsolete; the debugging and optimization work in parts (a) and (b) remains valuable as an exercise in correct key-scheduling logic and efficient block-cipher implementation technique, but should not be the basis of a production system going forward.

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

- A. Debug incorrect padding and IV handling causing decryption failures.
- B. Refactor the code to minimize redundant computations and memory usage.
- C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

Solution: A Python script implements AES-like block cipher modes (ECB, CBC, CFB, OFB) but suffers from high latency and memory inefficiency. (a) Debug incorrect padding and IV handling causing decryption failures. (b) Refactor the code to minimize redundant computations and memory usage. (c) Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

(a) Debugging the Padding and IV Handling

Buggy encrypt/decrypt routines:

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
import os

def encrypt(plaintext, key):
    iv = os.urandom(16)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    padded = pad(plaintext, 8)          # BUG 1: wrong block size (8, leftover from DES)
    return cipher.encrypt(padded)      # BUG 2: iv computed but never
    returned/stored

def decrypt(ciphertext, key):
    iv = os.urandom(16)                # BUG 3: generates a NEW random IV, not the
    real one
    cipher = AES.new(key, AES.MODE_CBC, iv)
    padded = cipher.decrypt(ciphertext)
    return unpad(padded, 8)             # BUG 1 (again): wrong block size
```

Bug 1 — Wrong block size passed to pad()/unpad(): AES operates on 16-byte blocks, but the padding calls use 8 (the DES/3DES block size, apparently left over from adapting the DES code in Question 1). PKCS#7 padding computed against the wrong block size produces a padded length that is not a multiple of AES's actual 16-byte block size, and, symmetrically, unpad() validates the decrypted trailing bytes against the wrong block size — both defects independently cause AES.new(...).encrypt()/decrypt() to raise a ValueError or to strip the wrong number of trailing bytes, corrupting the recovered plaintext.

Bug 2 — The IV is generated but discarded: `encrypt()` creates a fresh, correctly random iv, uses it to construct the cipher object, but then returns only `cipher.encrypt(padded)` — the IV value itself is never returned to the caller, written to the output, or stored anywhere. Once the function returns, that IV is permanently lost.

Bug 3 — `decrypt()` invents its own new random IV: Because the true IV was lost (Bug 2), `decrypt()` cannot use it — and rather than surfacing this as a missing argument, the buggy code silently generates a brand-new, unrelated random IV with `os.urandom(16)`. In CBC mode, the first decrypted plaintext block is computed as $D_K(C_1) \oplus IV$, so decrypting with the wrong IV guarantees the first 16 bytes of 'recovered plaintext' are garbage on every single decryption attempt, and the resulting corrupted padding bytes then fail `unpad()`'s validation — this combination is exactly the 'decryption failures' described in the task.

Fixed encrypt/decrypt routines:

```
def encrypt(plaintext, key):
    iv = os.urandom(16)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    padded = pad(plaintext, AES.block_size)    # FIX 1: correct 16-byte block size
    ciphertext = cipher.encrypt(padded)
    return iv + ciphertext                    # FIX 2: prepend the IV to the output

def decrypt(data, key):
    iv, ciphertext = data[:16], data[16:]      # FIX 3: recover the IV actually used
    cipher = AES.new(key, AES.MODE_CBC, iv)
    padded = cipher.decrypt(ciphertext)
    return unpad(padded, AES.block_size)      # FIX 1 (again): matching block size
```

The corrected design follows standard practice: the IV does not need to be secret (only unpredictable and never reused with the same key), so it is safe to transmit it in the clear by simply prepending it to the ciphertext; the receiver then only needs the shared key to slice the IV back off the front of the received data and decrypt correctly. The same `pad(..., AES.block_size)/unpad(..., AES.block_size)` pairing must be applied consistently everywhere the code pads or un pads data — using two different block-size constants on the encrypt and decrypt sides, as in the buggy version, is on its own sufficient to break every decryption.

(b) Refactoring to Minimize Redundant Computation and Memory Usage

Inefficient file-encryption routine:

```
def encrypt_file_slow(filename, key, iv):
    with open(filename, 'rb') as f:
        data = f.read()                    # loads entire file into RAM
    result = b''
    for i in range(0, len(data), 16):
        block = data[i:i+16]
        cipher = AES.new(key, AES.MODE_CBC, iv)    # BUG: re-instantiated every
    block
        result += cipher.encrypt(pad(block, 16))    # BUG: every block padded
    individually
    return result
```

- Whole-file loading: reading the entire input file into memory before encryption begins means peak memory usage scales linearly with file size, which is unnecessary for a task that only ever needs to look at a small sliding window of data at a time.
- Cipher object re-instantiated per block using the same static iv: constructing a brand-new `AES.new(key, AES.MODE_CBC, iv)` object for every 16-byte block, always with the same starting iv variable, silently discards CBC's chaining state between blocks — this is not merely inefficient, it also breaks CBC's actual security property, since every block is effectively encrypted as if it were the very first block of an independent message under a constant IV (functionally closer to a broken variant of ECB than to real CBC).
- Every block individually padded: calling `pad(block, 16)` inside the loop pads every single 16-byte block up to a full extra block (since PKCS#7 padding always adds at least 1 byte, a full 16 bytes if the input is already block-aligned), rather than padding only the final partial block of the entire message — this both corrupts the byte stream once properly unpadded and needlessly roughly doubles the amount of data actually encrypted and written out.
- $O(n^2)$ result accumulation: `result += cipher.encrypt(...)` repeatedly concatenates onto a Python bytes object; because bytes are immutable, each `+=` allocates an entirely new buffer and copies all previously accumulated data into it, making total accumulation cost quadratic in the number of blocks for a large file.

Refactored, streaming file-encryption routine:

```
def encrypt_file_fast(in_path, out_path, key, chunk_size=1 << 20):
    iv = os.urandom(16)
    cipher = AES.new(key, AES.MODE_CBC, iv)    # instantiated ONCE; internal
    chaining state persists
    with open(in_path, 'rb') as fin, open(out_path, 'wb') as fout:
        fout.write(iv)
        buf = bytearray()
        while chunk := fin.read(chunk_size):    # stream in 1 MB chunks, bounded
memory
            buf.extend(chunk)
            n_full = (len(buf) // 16) * 16
            if n_full:
                fout.write(cipher.encrypt(bytes(buf[:n_full])))
                del buf[:n_full]
            fout.write(cipher.encrypt(pad(bytes(buf), 16))) # pad ONLY the final partial block
```

The refactored version fixes all four issues at once: a single AES cipher object is created and reused for the entire file, so CBC's chaining state correctly carries forward from block to block exactly as the mode requires; the file is streamed through a fixed-size buffer rather than loaded wholesale, bounding memory usage independently of file size; padding is applied exactly once, only to the final partial block; and output is written incrementally to disk via `fout.write()` rather than accumulated in memory with repeated bytes concatenation, avoiding both the quadratic-time accumulation cost and the need to hold the entire ciphertext in RAM simultaneously.

(c) Comparing and Optimizing Modes for Real-Time Secure Communication

Mode	Padding Required?	Parallelizable (Encrypt / Decrypt)	Error Propagation	Real-Time Suitability
ECB	Yes	Yes / Yes	Single block only	Fast but insecure — identical blocks leak patterns (see Question 2 of Assessment 1); unsuitable regardless of speed
CBC	Yes	No / Yes	Current block + next block corrupted	Sequential encryption plus per-message padding both add latency; least suited to real-time streaming of the four
CFB	No	No / Yes	Current block + next block corrupted (self-synchronizing after)	No padding overhead; can encrypt arbitrary-length streams, but keystream for each block still depends on the previous ciphertext, so encryption remains sequential
OFB	No	No / No (but keystream precomputable)	Only the corresponding bits of that block — no cascading	No padding; keystream depends only on key + IV (not on ciphertext), so it can be generated ahead of time and applied via a single fast XOR the instant data arrives

Among the four modes the task asks about, OFB (Output Feedback) is the most efficient choice for real-time secure communication. Two properties drive this conclusion. First, like CFB, OFB eliminates the padding step entirely — the keystream is generated block-by-block and XORed directly with plaintext of any length, so a real-time system never has to wait to accumulate a full block, nor pay the (small but nonzero) latency and bandwidth cost of padding bytes, unlike ECB and CBC. Second, and more importantly for latency, OFB's keystream ($E_K(IV)$, $E_K(E_K(IV))$, ...) depends only on the key and IV — never on the plaintext or ciphertext — so the entire keystream for a session can be computed in advance, before any actual data has even arrived, and applied to incoming data with nothing more than a single XOR operation per byte the moment it becomes available. CFB, by contrast, feeds the previous ciphertext block back into the next keystream computation, so its keystream cannot be precomputed and encryption remains inherently sequential, adding latency that OFB avoids.

CBC is the least suitable of the four for real-time use: it requires padding (adding both computation and a data-dependent latency step at the end of each message), and — as demonstrated in the refactor of part (b) — its block-to-block chaining makes encryption strictly sequential, so no block can be encrypted until the previous block's ciphertext is available, which directly conflicts with the low, predictable latency that real-time communication demands. ECB, while nominally the fastest of the four because every block

is independent and trivially parallelizable, must be excluded on security grounds alone regardless of its raw speed, since it leaks repeated plaintext structure directly into the ciphertext, as analyzed in Question 2 of the previous assessment.

A final optimization note beyond the four modes listed in the task: in current real-world practice, CTR (Counter) mode — and its authenticated extension, GCM — is generally preferred over OFB for new designs, because CTR shares OFB's precomputable-keystream and no-padding advantages while additionally allowing every block to be generated independently (not just precomputed sequentially), enabling full parallel encryption/decryption across multiple CPU cores and random access into the middle of a stream without decrypting everything before it. For the specific comparison requested in this task, however, OFB should be identified as the most efficient of the four listed modes (ECB, CBC, CFB, OFB) for real-time secure communication, with CTR/GCM noted as the recommended modern evolution of the same underlying idea for any new system design.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

